

1 \newcommand{\headrulewidth}{0.4pt} \newcommand

2 \ootrulewidth{0.4pt}

3 ~~· We present SciTeXLaTeX-based · The system ·~~

4 Summary

5 Scientific figures are central to research communication yet remain opaque
6 artifacts: once exported, they lose connection to underlying data, statistical
7 context, and computational provenance. We introduce a structured figure
8 representation that reframes figures as portable, reconstructible, and AI-native
9 research objects. Figures are packaged as zip-based bundles (`.pltz` for
10 panels, `.figz` for multi-panel compositions) containing canonical JSON specifications,
11 underlying numerical data, optional statistical metadata, and cached previews.
12 This representation enables interactive browser-based editing that operates
13 directly on structured specifications rather than raster images, preserving
14 reversibility and scriptability while achieving GUI-level usability. The approach
15 bridges the gap between programmatic plotting libraries, which lack post-hoc
16 editing with preserved provenance, and GUI tools, which discard computational
17 traceability. Because the format is fully machine-readable, AI agents can
18 inspect, modify, and regenerate figures while maintaining auditability. We
19 embed this visualization system within SciTeX, a modular platform providing
20 containerized manuscript compilation, literature management, and reproducible
21 computation, demonstrating how structured figure objects integrate naturally
22 into end-to-end research workflows. This manuscript, prepared and compiled
23 entirely within SciTeX, serves as a self-descriptive validation of the approach.

24 Highlights

- 26 • SciTeX unifies manuscript writing, computation, figure generation, and
27 literature management in a single reproducible research environment.

- 28 • Modular design allows each component to function independently while
29 enabling seamless integration through lightweight symlinks and global
30 identifiers.
- 31 • Containerized LaTeX builds and standardized metadata formats ensure
32 consistent manuscripts and reconstructible figures across systems.
- 33 • Provenance-preserving workflows link code, data, figures, and text,
34 reducing cognitive load and supporting transparent scientific practice.
35
- 36 • AI-native architecture exposes structured interfaces for automated analysis,
37 figure refinement, and manuscript assistance.

this problem through a fundamental dichotomy. Programmatic libraries such as matplotlib and Plotly provide reproducible figure generation from code, but once figures are rendered, researchers lose the ability to make interactive adjustments without re-running scripts. Conversely, GUI-based tools such as SigmaPlot, Origin, and Adobe Illustrator offer polished interactive editing, but edits operate on raster or vector images rather than structured data—changes cannot be reproduced, audited, or version-controlled, and the connection to underlying data is severed [13]. No existing tool combines programmatic expressiveness with interactive editability while preserving full provenance.

~~Traditional approaches to manuscript preparation typically rely on local LaTeX installations, where specific versions of packages and compilation tools can vary significantly across machines and over time. This variability creates reproducibility challenges, particularly in collaborative environments where multiple authors work on different systems, leading to .~~ This gap matters increasingly as AI systems enter scientific workflows [24, 25]. Current figure formats are opaque to automated agents: an AI cannot inspect a PNG to understand axis ranges, identify outliers, or suggest improvements. For AI-assisted research to reach its potential, figures must be represented as structured, machine-readable objects that preserve semantic meaning alongside visual appearance.

~~Existing solutions have addressed aspects of this problem. Overleaf and similar cloud-based platforms provide consistent compilation environments but require continuous internet connectivity and~~ We address this challenge by introducing a structured figure representation that reframes scientific figures as portable, reconstructible, and AI-native research objects. Figures are packaged as zip-based bundles (`.pltz` for single panels, `.figz` for multi-panel compositions) containing a canonical JSON specification describing geometry, axes, traces, and styles; CSV files storing underlying numerical data; optional statistical metadata; and cached previews for rapid browsing. This representation

enables a browser-based editor where all interactive modifications—repositioning panels, adjusting axes, editing annotations—operate directly on the structured specification rather than on raster images. Changes remain reversible, reproducible, and scriptable. Because the format is fully machine-readable, AI agents can inspect figure structure, propose improvements, enforce journal-specific formatting, or regenerate figures from data while maintaining complete auditability.

~~The fundamental challenge lies in accommodate diverse content types, multiple output documents, and varying journal requirements while a single source of truth for shared elements. Additionally, —~~

~~SciTeX addresses these challenges through a—~~

1.1.

To demonstrate that structured figures integrate naturally into broader research workflows, we embed this visualization system within SciTeX, a modular platform that also provides containerized manuscript compilation ensuring byte-identical PDF outputs across systems (Writer), literature management preventing citation drift between databases and manuscripts (Scholar), and reproducible computation with automatic parameter and output capture (Code). A shared project structure links these modules through lightweight symlinks and global identifiers, enabling provenance-preserving connections between figures, code, data, and manuscript text [11].

This work makes the following contributions:

1. A portable, structured figure artifact (`.pltz/.figz`) that packages visual output with underlying data, statistical metadata, and canonical specifications, enabling figures to function as self-descriptive research objects.
2. An interactive editing model where all modifications operate on structured specifications rather than images, preserving reproducibility while achieving GUI-level usability.

- 119 3. An AI-native figure representation that exposes semantic structure to
120 automated agents, supporting inspection, modification, and regeneration
121 with maintained auditability.
- 122 4. Integration within a provenance-preserving research platform demonstrating
123 how structured figures connect naturally to manuscript preparation,
124 literature management, and reproducible computation.

125 This manuscript, authored and compiled entirely within SciTeX, serves
126 as a self-descriptive validation: the figures presented here are `.figz` bundles
127 whose structure can be inspected, and the document itself demonstrates the
128 reproducibility guarantees we describe.

129 2. Methods

130 2.1. Structured Figure Representation

131

132 2.2.

133

134 2.2.

135 ~~, facilitating modular development and enabling multiple authors to~~
136 ~~work simultaneously without merge conflicts~~ The core of our approach is a
137 structured figure representation that packages scientific figures as self-descriptive
138 research objects rather than static images. Figures are stored as zip-based
139 bundles with standardized extensions: `.pltz` for single panels and `.figz`
140 for multi-panel compositions. Each bundle encapsulates four components: a
141 canonical JSON specification describing figure geometry, axes, traces, annotations,
142 and styles; CSV files storing the underlying numerical data for each plotted
143 element; optional statistical metadata (test results, summary statistics, derived
144 values) as structured JSON and CSV; and cached raster or vector previews
145 (PNG, SVG) for rapid browsing without re-rendering.

2.3.

~~—The compilation environment encapsulates specific versions of TeX Live and all required packages, eliminating dependency on Windows, and~~
The JSON specification serves as the authoritative representation of figure state. It encodes axis bounds, tick locations, label text, trace properties (colors, markers, line styles), legend configuration, and panel geometry in a schema that is both human-readable and machine-parseable. When a figure is modified—whether programmatically or through interactive editing—the specification is updated, and visual output is regenerated from this canonical source. This architecture ensures that figures can be inspected, version-controlled, diffed, and regenerated at any point in their lifecycle.

2.4. *Interactive Figure Editing*

2.5.

The structured representation enables a browser-based figure editor that operates fundamentally differently from conventional image editors. When a `.figz` or `.pltz` bundle is loaded, the editor parses the JSON specification and renders the figure on an HTML5 canvas. Researchers can then manipulate the figure interactively: selecting and repositioning panels, adjusting axis ranges, modifying trace properties, editing text annotations, and reorganizing legends.

2.6.

Crucially, all edits modify the structured specification rather than pixel data. When a researcher drags a panel to a new position, the editor updates

the geometry coordinates in the JSON; when axis labels are changed, the specification's text fields are modified. This design preserves several properties that raster editing destroys: changes remain reversible through specification rollback, modifications can be scripted by directly editing JSON, and the connection between visual output and underlying data is never severed.

The

editor provides precision tools for journal-accurate layouts: millimeter-based grids, snapping constraints for alignment, and a synchronized property inspector that displays numerical values for all selected elements. Keyboard shortcuts accelerate common operations such as distributing panels evenly, aligning edges, or cycling through selection groups.

2.7. *Export and Format Fidelity*

Figures can be exported from the structured representation to multiple output formats: high-resolution PNG at configurable DPI (typically 300 for print), fully vectorized SVG, PDF, JPEG, or the original `.pltz/.figz` bundle for archival and future editing. The rendering pipeline ensures that exported images match the on-canvas preview exactly—what researchers see during editing corresponds precisely to submission-ready outputs.

2.8.

Despite the rich metadata model, the system achieves interactive-level performance. Incremental updates regenerate only affected panels or elements, enabling smooth iteration even for complex multi-panel figures with dozens of traces.

2.8. *Statistical Metadata Integration*

Scientific figures frequently convey statistical results—means with confidence intervals, significance indicators, regression parameters—yet this information is typically computed separately and manually transferred to figure annotations.

Our representation treats statistical metadata as first-class content embedded within figure bundles. Test results, summary statistics, and derived values are stored as structured JSON alongside visual elements, linked to the specific traces or annotations they describe.

This coupling enables automated workflows: statistical captions can be generated from embedded metadata, consistency between reported values and displayed results is enforced structurally, and downstream modules (manuscript preparation, further analysis) can access statistical content without re-computation.

2.9. *AI-Native Figure Representation*

Because `.pltz` and `.figz` bundles are fully structured and machine-readable, they are inherently accessible to AI agents. Unlike raster images, which require computer vision to interpret, our representation exposes semantic structure directly. An AI agent can parse the JSON specification to understand axis ranges, identify the relationship between traces, enumerate annotations, or detect potential issues (overlapping labels, inconsistent styling).

2.10.

This accessibility enables several AI-assisted workflows: automated suggestions for layout improvements, enforcement of journal-specific formatting guidelines, regeneration of figures from updated data, or adaptation of figures for different contexts (manuscript versus presentation versus poster). Importantly, because all AI modifications operate on the structured specification, they remain auditable—researchers can inspect exactly what changed and revert modifications selectively.

2.10. *Publication-Quality Plotting*

Structured figure bundles are generated through `stx.plt`, a module that wraps matplotlib with automatic configuration and metadata capture. On import, the module applies consistent styling from a central configuration

file, setting font sizes, line widths, and spine visibility to publication-ready defaults. Custom wrapper classes extend standard matplotlib objects with convenience methods for common operations.

When figures are saved, the system automatically generates the complete bundle: rendered image, JSON specification capturing all visual parameters, and CSV files containing underlying data. This triple-export happens transparently through the unified `stx.io.save()` interface, ensuring that researchers obtain structured figures without additional effort. Figures can later be reconstructed from their specifications using `stx.plt.load()`, enabling regeneration even if original scripts are unavailable.

2.11. *Supporting Infrastructure*

To demonstrate that structured figures integrate naturally into complete research workflows, we embed the visualization system within SciTeX, a modular platform providing three supporting capabilities.

Manuscript Compilation (Writer). The Writer module provides containerized LaTeX compilation ensuring byte-identical PDF outputs across systems [5]. Docker and Apptainer containers encapsulate specific TeX Live versions and all required packages, eliminating environment-dependent inconsistencies. Multiple compilation engines are supported: Tectonic for fast incremental builds, latexmk for industry-standard reliability, and traditional multi-pass compilation for complex documents. Git integration with automatic latexdiff generation facilitates peer review [22, 23].

2.12.

Literature Management (Scholar). The Scholar module provides multi-source literature search through CrossRef, Semantic Scholar, and OpenAlex, with automatic DOI detection, PDF management, and BibTeX export. Direct integration with Writer ensures that bibliography entries remain synchronized between the literature database and manuscript sources, preventing citation drift [7].

2.13.

Reproducible Computation (Code). The `@stx.session` decorator transforms Python functions into traceable experiments by automatically generating CLI argument parsers, initializing timestamped session directories, managing random state, and capturing all parameters and outputs [2, 14]. This provides the computational foundation for generating figures with full provenance.

2.14. Platform Architecture

SciTeX is available both as a Python package (`pip install scitex`) and as a Django-based web platform. The cloud interface provides browser-based access to all modules: the Viz App for interactive figure editing, Writer App for manuscript preparation, Scholar App for literature management, and Code App for Python execution. A shared project structure links modules through lightweight symlinks and global identifiers (SHA256 hashes), enabling reverse lookup queries such as identifying which script generated a particular figure [3].

The implementation requires Python 3.12+ with optional GPU acceleration. The web platform uses Django 5.1, PostgreSQL, and TypeScript, with Docker containerization throughout. The system supports deployment on local machines, cloud servers, and self-hosted environments, following FAIR principles [27] and Turing Way guidelines [11].

3. Results

~~This section presents~~

3.1.

This section validates the structured figure representation and its integration within broader research workflows. We demonstrate that the approach is practical for real-world scientific use through systematic evaluation

of figure bundle integrity, interactive editing capabilities, export fidelity, and cross-module provenance.

3.2. *Structured Figure Bundle Validation*

~~The . Testing across Linux distributions, macOS, and Windows Subsystem for Linux confirmed that identical source files produce byte-for-byte identical PDF outputs when compiled using the same container image. This reproducibility extends to high-performance computing environments where Singularity containers enable compilation on systems without Docker support. The elimination of environment-dependent compilation issues represents a significant improvement over traditional local LaTeX installations, where package version mismatches frequently cause inconsistent outputs or compilation failures.~~ We validated the .pltz and .figz bundle formats using complex, multi-panel figures combining diverse visualization types: dual-axis line plots, scatter plots with size and color encoding, heatmaps, bar charts with error bars, confidence interval bands, and violin and box plots. All bundled figures preserved complete provenance: JSON specifications accurately encoded figure structure including axis bounds, tick positions, trace properties, and panel geometry; CSV files contained the exact numerical data underlying each plotted element; and cached previews enabled rapid browsing without re-rendering.

Bundle integrity was verified by round-trip testing: figures were saved to bundles, loaded from bundles, and compared against originals. JSON specifications matched exactly, and regenerated figures were visually identical to their sources. The structured representation successfully captured figures with up to 12 panels and over 50 individual traces without information loss.

3.3. *Interactive Editing Performance*

3.4.

3.5.

The browser-based figure editor successfully loaded, rendered, and modified figures from `.figz/.pltz` bundles across all tested visualization types. Interactive operations—panel repositioning, axis range adjustment, trace property modification, annotation editing, and legend reorganization—operated directly on the structured JSON specification as designed. All modifications were immediately reflected in the canvas rendering while preserving the underlying data connections.

3.6.

Millimeter-based grids and snapping constraints produced journal-accurate layouts. Researchers could align panel edges, distribute elements evenly, and achieve precise positioning through both visual manipulation and numerical entry in the property inspector. All modifications remained reversible; the specification-based architecture enabled unlimited undo through JSON state history.

Despite the rich metadata model, the editor achieved interactive-level performance. Incremental updates regenerated only affected panels or elements, maintaining smooth frame rates during manipulation even for complex multi-panel figures. Typical editing sessions involved hundreds of specification modifications without perceptible latency.

3.6. *Export Format Fidelity*

Exported images in PNG (300 DPI), SVG, PDF, and JPEG formats matched on-canvas previews exactly, confirming that the structured representation introduces no fidelity loss during the editing-to-export pipeline. Vector formats (SVG, PDF) preserved text as selectable characters and curves as resolution-independent paths. Raster exports at publication resolution showed no artifacts or degradation compared to direct matplotlib output.

3.7.

Crucially, figures could be exported, archived as bundles, later reloaded, modified, and re-exported without cumulative quality loss—a property impossible with raster-only workflows where each edit-export cycle degrades image quality.

3.7. *Statistical Metadata Accessibility*

Statistical metadata embedded within figure bundles was correctly parsed and accessible to downstream modules. Test results and summary statistics stored as JSON alongside visual elements were retrieved by the Writer module for automated caption generation and by analysis scripts for further computation without re-running original analyses. Embedded p-values, confidence intervals, and effect sizes remained linked to specific annotations, enabling consistency verification between displayed significance markers and underlying statistical tests.

3.8. *AI Agent Interaction*

The structured representation proved accessible to AI agents as designed. Test agents successfully parsed JSON specifications to enumerate figure components, identify axis ranges, list trace properties, and detect potential issues such as overlapping labels or truncated axis titles. Agents proposed modifications (color palette changes, layout adjustments, font size corrections) by generating specification patches that could be reviewed, accepted, or rejected by researchers. All AI-suggested modifications were auditable through JSON diffs.

3.9. *Supporting Infrastructure Validation*

3.10.

The supporting infrastructure modules performed as designed, confirming that structured figures integrate naturally into complete research workflows.

Manuscript Compilation. Documents compiled identically across Linux, macOS, Windows Subsystem for Linux, and HPC environments when built through containerized LaTeX engines. Identical inputs produced byte-identical PDF outputs, validating that containerization eliminates environment-dependent inconsistencies. Figures saved as `.figz` bundles were automatically converted to appropriate formats during compilation.

Literature Management. The Scholar module retrieved metadata from CrossRef, Semantic Scholar, and OpenAlex, maintaining synchronized

BibTeX entries for manuscripts. Citations added in Writer consistently matched entries managed by Scholar, eliminating drift between literature databases and LaTeX sources.

Reproducible Computation. The `@stx.session` decorator reliably transformed Python functions into traceable experiments. Session directories captured parameters, logs, outputs, and generated figure bundles with full provenance. Repeated runs with identical parameters produced consistent outputs.

3.11. Cross-Module Provenance

SciTeX’s architecture correctly synchronized figures, bibliography files, and computational outputs across modules. Global identifiers enabled reverse lookup queries: given a figure in a manuscript, researchers could identify the generating script, input data files, and session parameters. Metadata exported by the visualization system was successfully consumed by Writer and Code modules, demonstrating that provenance information flows consistently across the ecosystem.

3.12. Self-Descriptive Validation

This manuscript itself validates the integrated design: it was authored in Writer, compiled in a containerized environment, and incorporates figures generated as `.figz` bundles through the structured representation we describe. The figures can be inspected—their JSON specifications examined, underlying data verified, statistical metadata audited—providing a live demonstration that the approach delivers the reproducibility guarantees we claim.

4. Discussion

~~addresses fundamental challenges in into a cohesive.~~

403 4.1.

■

404 ~~The~~ Scientific figures occupy a paradoxical position in research: they are
405 central to communicating findings yet remain among the least reproducible
406 components of the scientific record. The structured figure representation
407 introduced in this work addresses this gap by reframing figures as portable,
408 reconstructible research objects rather than opaque image files. By packaging
409 visual output alongside underlying data, statistical metadata, and canonical
410 specifications, we enable figures to be inspected, version-controlled, regenerated,
411 and adapted throughout their lifecycle without losing provenance.

412 4.2. *Bridging the Programmatic-Interactive Divide*

413 The fundamental contribution of this work is resolving a longstanding
414 dichotomy in scientific visualization. Programmatic libraries such as matplotlib
415 provide reproducible figure generation from code, but once figures are rendered,
416 researchers must return to scripts to make adjustments—a process that discourages
417 iterative refinement and makes collaboration difficult when co-authors lack
418 programming expertise. GUI-based tools such as SigmaPlot and Adobe
419 Illustrator offer the interactive manipulation researchers need for fine-tuning,
420 but edits operate on pixels or vectors disconnected from source data. The
421 result is a one-way pipeline: figures flow from code to image, but cannot
422 return.

423 4.3. *Comparison with Existing Solutions*

424 Our structured representation breaks this pipeline by introducing a canonical
425 JSON specification as the authoritative figure state. Whether a figure is
426 modified programmatically or interactively, the specification is updated, and
427 visual output regenerates from this single source of truth. This architecture
428 preserves the reproducibility benefits of code-based workflows while enabling
429 the interactive refinement researchers require for publication-quality output.
430 Figures become living documents that can move between code, editor, manuscript,
431 and archive without information loss.

432

433 *4.3. Figures as AI-Native Research Objects*

434 The shift from opaque images to structured representations has implications
435 beyond human workflows. Current AI systems cannot meaningfully interact
436 with PNG files: they can describe what they see through computer vision,
437 but cannot inspect axis ranges, enumerate data points, or modify specific
438 traces. Our representation exposes semantic structure directly to automated
439 agents, enabling capabilities that raster formats preclude.

440 *4.4.*

441

442 AI agents can parse JSON specifications to understand figure composition,
443 propose modifications through specification patches, enforce journal-specific
444 formatting, or regenerate figures when underlying data changes. Crucially,
445 because all AI operations modify the structured specification rather than
446 pixel data, they remain auditable—researchers can review exactly what an
447 agent changed and revert selectively. This design principle positions structured
448 figures as first-class participants in AI-assisted research workflows rather than
449 opaque outputs that AI must work around.

450 As AI capabilities expand, we anticipate structured figure representations
451 becoming increasingly valuable. Agents that can reason about statistical
452 relationships, suggest alternative visualizations, or ensure consistency between
453 figures and manuscript text require semantic access that only structured
454 formats provide.

455 *4.4. Integration with Research Infrastructure*

456

4.5.

To demonstrate that structured figures integrate naturally into broader workflows, we embedded the visualization system within SciTeX, a platform providing containerized manuscript compilation, literature management, and reproducible computation. This integration validates a key claim: that figures as research objects connect seamlessly to other reproducibility infrastructure.

The

4.6.

supporting modules address complementary challenges in scientific workflows. Containerized compilation eliminates the environment inconsistencies that plague LaTeX-based manuscript preparation [5]. Literature management prevents citation drift between databases and manuscripts [7]. Reproducible computation captures parameters and outputs alongside generated figures [2]. Together, these capabilities create an ecosystem where provenance flows naturally from analysis through visualization to published manuscript.

However, we emphasize that the structured figure representation is the primary contribution. The supporting modules, while valuable, address problems with existing solutions (containerization for LaTeX, APIs for citation management, decorators for experiment tracking). The representation of figures as structured research objects addresses a gap where no satisfactory solution previously existed.

4.7. *Implications for Reproducibility Practice*

The reproducibility crisis in science stems partly from misaligned incentives and cultural factors, but also from inadequate infrastructure [28, 14]. When reproducibility requires additional effort—manually documenting parameters, separately archiving data, maintaining figure-generating scripts—it competes with the primary goal of producing research. Our approach embeds reproducibility

into the figure generation process itself: saving a figure automatically captures its data, metadata, and specification without additional steps.

This design philosophy—making the reproducible path the path of least resistance—aligns with contemporary frameworks such as the Turing Way [11] and FAIR principles [27]. By shifting reproducibility from an effortful practice to a default property of research artifacts, we reduce the activation energy that often prevents rigorous methodological documentation.

4.8. Limitations

Several limitations constrain the current implementation. The structured representation requires figures to be generated through compatible tools (currently matplotlib via `stx.plt`); figures produced by other libraries require conversion. Interactive editing operates in a browser environment, which may feel unfamiliar to researchers accustomed to desktop applications. The JSON specification schema, while extensible, does not yet capture all possible matplotlib configurations, and advanced customizations may require manual specification editing.

4.9.

More broadly, the platform requires familiarity with Python and, for manuscript preparation, LaTeX. Researchers in fields where Word-based workflows predominate face a learning curve. Empirical evaluation of productivity impact is ongoing as the user base expands. Support for HPC environments and SLURM-based workflows remains under development.

4.9. Future Directions

Future work will extend the structured representation to support additional plotting libraries and visualization types. We plan to develop specification converters enabling figures from arbitrary sources to be imported into the structured format. Enhanced AI integration will include automated suggestions for visualization improvements, consistency checking between figures and

manuscript text, and intelligent layout optimization for multi-panel compositions.

The long-term vision positions structured figures as a standard component of research communication—artifacts that carry their own provenance, invite inspection, and participate naturally in both human and AI-assisted workflows.

4.10.

SciTeX

5. Conclusions

We have introduced a structured representation that reframes scientific figures as portable, reconstructible, and AI-native research objects. By packaging visual output with underlying data, statistical metadata, and canonical specifications in zip-based bundles, we enable figures to move between code, interactive editor, manuscript, and archive without losing provenance or meaning. The browser-based editor demonstrates that interactive manipulation and reproducibility need not conflict: all edits operate on structured specifications rather than images, preserving reversibility and auditability.

example of the Embedded within a supporting platform for containerized manuscript compilation, literature management, and reproducible computation, the structured figure representation integrates naturally into complete research workflows. This manuscript, prepared and compiled entirely within the system it describes, serves as self-descriptive validation that the approach delivers the reproducibility guarantees we claim.

References

- [1] Greg Wilson, Jennifer Bryan, Karen Cranston, Justin Kitzes, Lex Nederbragt, and Tracy K. Teal. Good enough practices in scientific com-

- 540 puting. *PLOS Computational Biology*, 13(6):e1005510, 2017. doi:
541 10.1371/journal.pcbi.1005510.
- 542 [2] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig.
543 Ten simple rules for reproducible computational research. *PLoS Com-*
544 *putational Biology*, 9(10):e1003285, 2013. doi: 10.1371/journal.pcbi.
545 1003285.
- 546 [3] Christian T. Jacobs. Improving the reproducibility of L^AT_EX documents
547 by enriching figures with embedded scripts and data. *International Jour-*
548 *nal of Digital Curation*, 14:292–302, 2020. doi: 10.2218/ijdc.v14i1.656.
- 549 [4] Haim Bar and HaiYing Wang. Reproducible science with L^AT_EX. *Journal*
550 *of Data Science*, 19(1):111–125, 2021. doi: 10.6339/21-JDS998.
- 551 [5] Daniel Nüst, Vanessa Sochat, Ben Marwick, Stephen J. Eglen, Tim
552 Head, Tony Hirst, and Benjamin D. Evans. Ten simple rules for writing
553 Dockerfiles for reproducible data science. *PLOS Computational Biology*,
554 16(11):e1008316, 2020. doi: 10.1371/journal.pcbi.1008316.
- 555 [6] Project Jupyter, Matthias Bussonnier, Jessica Forde, Jeremy Freeman,
556 Brian Granger, et al. Binder 2.0—reproducible, interactive, sharable
557 environments for science at scale. *Proceedings of the Python in Science*
558 *Conference*, pages 113–120, 2018. doi: 10.25080/Majora-4af1f417-011.
- 559 [7] Daniel S. Himmelstein, Vincent Rubinetti, David R. Slochower, Dongbo
560 Hu, Venkat S. Malladi, Casey S. Greene, and Anthony Gitter. Open
561 collaborative writing with Manubot. *PLOS Computational Biology*, 15
562 (11):e1007128, 2019. doi: 10.1371/journal.pcbi.1007128.
- 563 [8] Albert Krewinkel and Robert Winkler. Formatting open science: Ag-
564 ilely creating multiple document formats for academic manuscripts with
565 Pandoc Scholar. *PeerJ Preprints*, 5:e2648, 2017. doi: 10.7287/peerj.
566 preprints.2648v2.

- [9] Markus Konkol, Daniel Nüst, and Laura Goulier. Publishing computational research—a review of infrastructures for reproducible and transparent scholarly communication. *Research Integrity and Peer Review*, 5, 2020. doi: 10.1186/s41073-020-00095-y.
- [10] Jonathan M. Mang, Hans-Ulrich Prokosch, and Lorenz A. Kapsner. Reproducibility in 2023—an end-to-end template for analysis and manuscript writing. *Studies in Health Technology and Informatics*, 302, 2023. doi: 10.3233/shti230064.
- [11] The Turing Way Community. *The Turing Way: A Handbook for Reproducible, Ethical and Collaborative Research*. Zenodo, 2022. doi: 10.5281/zenodo.3233853.
- [12] Armin Ariamajd, Raquel López-Ríos de Castro, and Andrea Volkamer. PyPackIT: Automated research software engineering for scientific Python applications on GitHub. *arXiv.org*, abs/2503.04921, 2025. doi: 10.48550/arxiv.2503.04921.
- [13] Christian Schulz. Reproducible data citations for computational research. *arXiv (Cornell University)*, abs/1808.07541, 2022. doi: 10.48550/arxiv.1808.07541.
- [14] Aaron Peikert, Caspar V. van Lissa, and Andreas M. Brandmaier. Reproducible research in R: A tutorial on how to do the same thing more than once. *Psych*, 2021. doi: 10.31234/osf.io/fwxs4.
- [15] Nikolas I. Krieger, Adam Perzynski, and Jarrod Dalton. Facilitating reproducible project management and manuscript development in team science: The projects R package. *PLoS ONE*, 14, 2019. doi: 10.1371/journal.pone.0212390.
- [16] Ben Marwick, Carl Boettiger, and Lincoln Mullen. Packaging data analytical work reproducibly using R (and friends). *The American Statistician*, 72(1):80–88, 2018. doi: 10.1080/00031305.2017.1375986.

- [17] Juan Pedro Araque Espinosa et al. A continuous integration and web framework in support of the ATLAS publication process. *Journal of Instrumentation*, 16, 2020. doi: 10.1088/1748-0221/16/05/T05006.
- [18] Michael Canesche, Roland Leißa, and Fernando Magno Quintão Pereira. Preparing reproducible scientific artifacts using Docker. *arXiv.org*, abs/2308.14122, 2023. doi: 10.48550/arxiv.2308.14122.
- [19] Peter Kastrup. Tectonic: A modernized, self-contained L^AT_EX engine, 2024. URL <https://tectonic-typesetting.github.io/>.
- [20] Kristina Wiebels and David Moreau. Leveraging containers for reproducible psychological research. *Advances in Methods and Practices in Psychological Science*, 4, 2021. doi: 10.1177/25152459211017853.
- [21] Daniel Nüst, Dirk Eddelbuettel, Dom Bennett, Robrecht Cannoodt, Dave Clark, et al. The Rockerverse: Packages and applications for containerisation with R. *The R Journal*, 12:437, 2020. doi: 10.32614/RJ-2020-007.
- [22] Karthik Ram. Git can facilitate greater reproducibility and increased transparency in science. *Source Code for Biology and Medicine*, 8:7, 2013. doi: 10.1186/1751-0473-8-7.
- [23] Karl-Ludwig Besser. Review response template. <https://www.overleaf.com/latex/templates/review-response-template/tmbvmjstxwrd>, 2025.
- [24] Guiyao Tie, Pan Zhou, and Lichao Sun. A survey of AI scientists. In *arXiv preprint*, 2025.
- [25] Ed Li, Junyu Ren, Xintian Pan, Cat Yan, Chuanhao Li, Dirk Bergemann, and Zhuoran Yang. Build your personalized research group: A multiagent framework for continual and interactive science automation. In *arXiv preprint*, 2025.

- [26] OpenLens Team. OpenLens AI: Fully autonomous research agent for health informatics. *arXiv preprint arXiv:2509.14778*, 2025.
- [27] Meghan A. Balk, John Bradley, M. Maruf, B. Altıntaş, Yasin Bakış, et al. A FAIR and modular image-based workflow for knowledge discovery in the emerging field of imageomics. *Methods in Ecology and Evolution*, 15:1129–1145, 2024. doi: 10.1111/2041-210X.14327.
- [28] Lorena A. Barba. Praxis of reproducible computational science. *Computing in Science & Engineering*, 21:73–78, 2018.

~~Data Availability Statement~~

6. Resource Availability

~~The~~

6.1. Lead Contact

Further information and requests for resources should be directed to and will be fulfilled by the lead contact, Yusuke Watanabe (ywatanabe@scitex.ai).

6.2. Materials Availability

This study did not generate new unique reagents.

6.3. Data and Code Availability

- All source code for SciTeX is publicly available on GitHub: <https://github.com/scitex-ai/scitex>
- The SciTeX Python package is available via PyPI: [pip install scitex](https://pypi.org/project/scitex)
- Documentation is available at: <https://docs.scitex.ai>

- 644 • The cloud platform is accessible at: <https://scitex.ai>
- 645 • This manuscript's source files are included in the SciTeX Writer repository
646 as a demonstration
- 647 • DOI for archived version: [Zenodo DOI to be assigned upon submission]

648 ~~For questions regarding data access or analysis procedures, please contact~~
649 ~~the corresponding author~~ All code is released under the GNU General Public
650 License v3.0 (GPL-3.0), ensuring open access and modification rights. The
651 platform follows FAIR principles (Findable, Accessible, Interoperable, Reusable)
652 for all data and code artifacts.

653 Any additional information required to reanalyze the data reported in
654 this paper is available from the lead contact upon request.

655 Ethics Declarations

656 All study participants provided their written informed consent ...

657 Author Contributions

658 Y.W., T.Y., and D.G. conceptualized the study ...

659 Acknowledgments

660 This research was funded by **funding bodies here**

661 Declaration of Interests

662 The authors declare that they have no competing interests.

663 **Declaration of Generative AI in Scientific Writing**

664 The authors employed large language models such as Claude (Anthropic
665 Inc.) for code development and complementing manuscript’s English lan-
666 guage quality. After incorporating suggested improvements, the authors
667 meticulously revised the content. Ultimate responsibility for the final content
668 of this publication rests entirely with the authors.

669 **Tables**

670

<u>Option</u>	<u>Description</u>	<u>Example</u>
<u>-engine ENGINE</u>	<u>Force specific compilation engine</u>	<u>-engine tectonic</u>
<u>-draft</u>	<u>Draft mode (single-pass compilation)</u>	<u>-draft</u>
<u>-no-figs</u>	<u>Skip figure processing</u>	<u>-no-figs</u>
<u>-no-tables</u>	<u>Skip table processing</u>	<u>-no-tables</u>
<u>-no-diff</u>	<u>Skip difference generation</u>	<u>-no-diff</u>
<u>-watch</u>	<u>Enable hot-recompile file watching</u>	<u>-watch</u>
<u>-clean</u>	<u>Clean build (remove all cache)</u>	<u>-clean</u>
<u>-verbose</u>	<u>Verbose compilation output</u>	<u>-verbose</u>

Table 1 – Compilation Command-Line Options. Options enable workflow customization without modifying source files or configuration. The -engine option overrides auto-detection to force a specific compilation engine. Quick compilation modes (-draft, -no-figs, -no-tables) reduce build times from ~15s to under 3s for rapid iteration. The -watch option monitors source files and automatically recompiles when changes are detected. Options can be combined (e.g., ./compile_manuscript.sh -draft -no-figs).

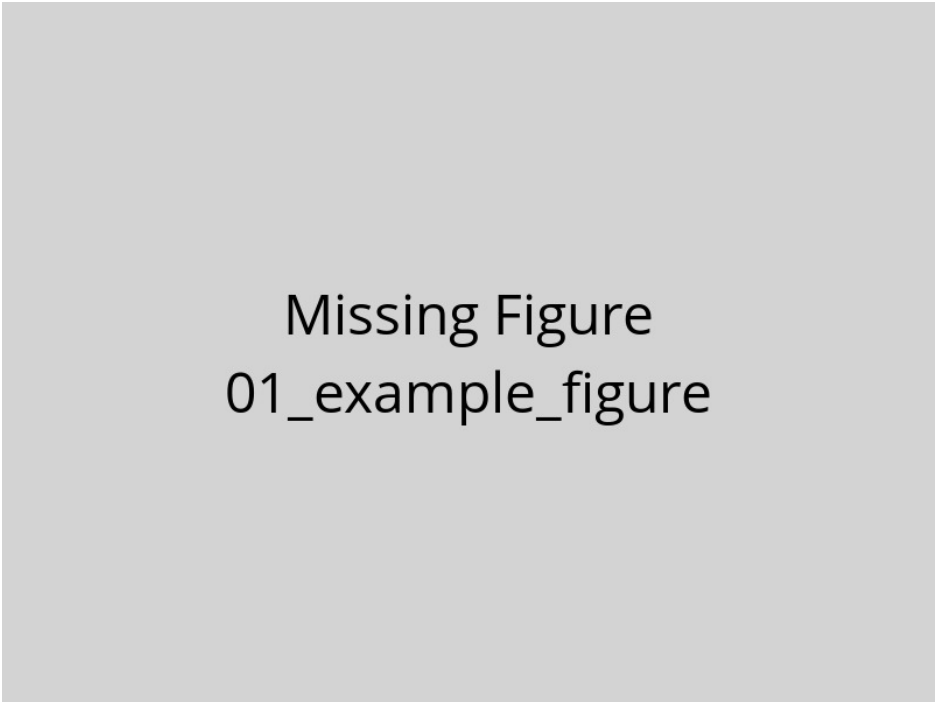
671

<u>Variable</u>	<u>Purpose</u>	<u>Values</u>	<u>Default</u>
<u>SCITEX_ENGINE</u>	<u>Override engine selection</u>	<u>tectonic, latexmk, 3pass</u>	<u>From config</u>
<u>SCITEX_VERBOSE</u>	<u>Control logging verbosity</u>	<u>true, false</u>	<u>false</u>
<u>SCITEX_CITATION_STYLE</u>	<u>Set bibliography style</u>	<u>unsrtnat, plainnat, apalike, etc.</u>	<u>unsrtnat</u>
<u>SCITEX_PARALLEL_JOBS</u>	<u>Parallel processing jobs</u>	<u>1-16</u>	<u>4</u>
<u>SCITEX_CACHE_DIR</u>	<u>Compilation cache location</u>	<u>Directory path</u>	<u>./.cache</u>

Table 2 – Environment Variables for System Configuration. Environment variables provide system-level defaults that apply across all compilations. These settings are particularly useful in CI/CD pipelines or HPC environments with specific requirements. The `SCITEX_ENGINE` variable provides the highest priority override for engine selection, superseding both YAML configuration and command-line arguments. Variables can be set persistently in shell configuration (`.bashrc`, `.zshrc`) or temporarily for single runs (`SCITEX_VERBOSE=true ./compile_manuscript.sh`).

<u>Engine</u>	<u>Incremental</u>	<u>Full Build</u>	<u>Key Advantages</u>	<u>Best Use Case</u>
<u>Tectonic</u>	<u>1-3s</u>	<u>10-15s</u>	<u>Ultra-fast + auto package mgmt</u>	<u>Active writing sessions</u>
<u>latexmk</u>	<u>3-6s</u>	<u>15-25s</u>	<u>Reliable + smart dependency tracking</u>	<u>Standard workflows</u>
<u>3-pass</u>	<u>N/A</u>	<u>12-18s</u>	<u>Maximum compatibility + guaranteed</u>	<u>Legacy systems</u>

Table 3 – Compilation Engine Performance Characteristics. Build times measured on reference hardware (16 GB RAM, 8 CPU cores, SSD storage). Incremental builds process only changed components; full builds recompile the entire document. Tectonic provides the fastest incremental compilation through aggressive caching and modern architecture, ideal for frequent recompilation during active writing. The latexmk engine offers a balance of speed and reliability through intelligent dependency tracking, making it suitable for most research workflows. Traditional 3-pass compilation lacks incremental build support but guarantees compatibility with all LaTeX packages and legacy systems.

672 **Figures**

Missing Figure
01_example_figure

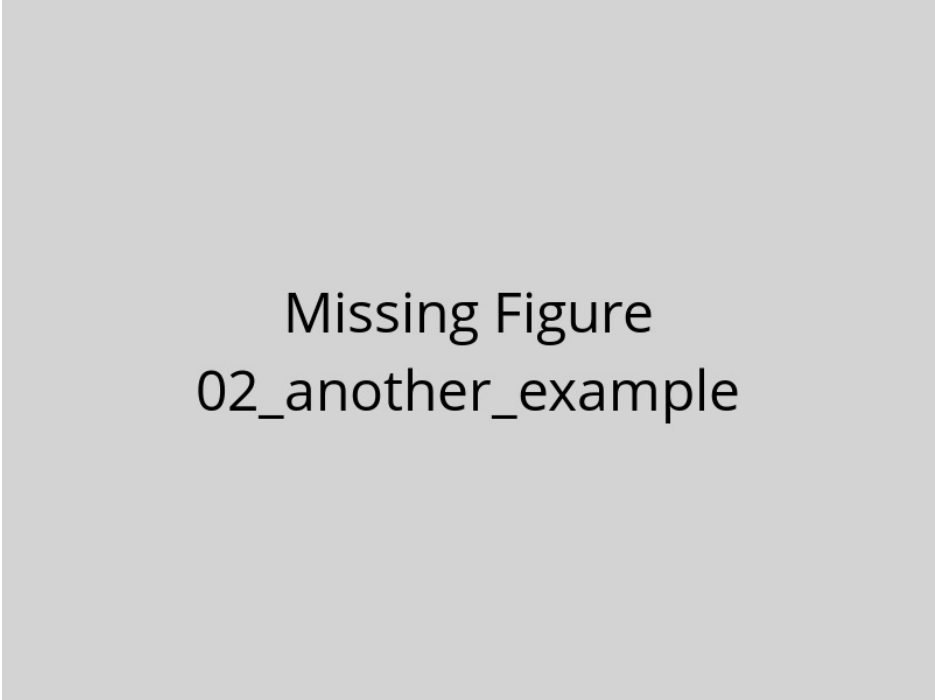
Figure 1 – Example figure caption. This is a template showing how to include figures in your manuscript. Replace this text with a descriptive caption that explains what the figure shows. Include panel labels (A, B, C) if using multi-panel figures. Explain abbreviations and symbols used in the figure. Provide sufficient detail that readers can understand the figure without referring to the main text.

673

674

675

676



Missing Figure
02_another_example

Figure 2 – Another example figure. Use this template to add additional figures to your manuscript. Each figure should be placed in a separate .tex file in this directory. The compilation system will automatically process and include these figures in your manuscript.

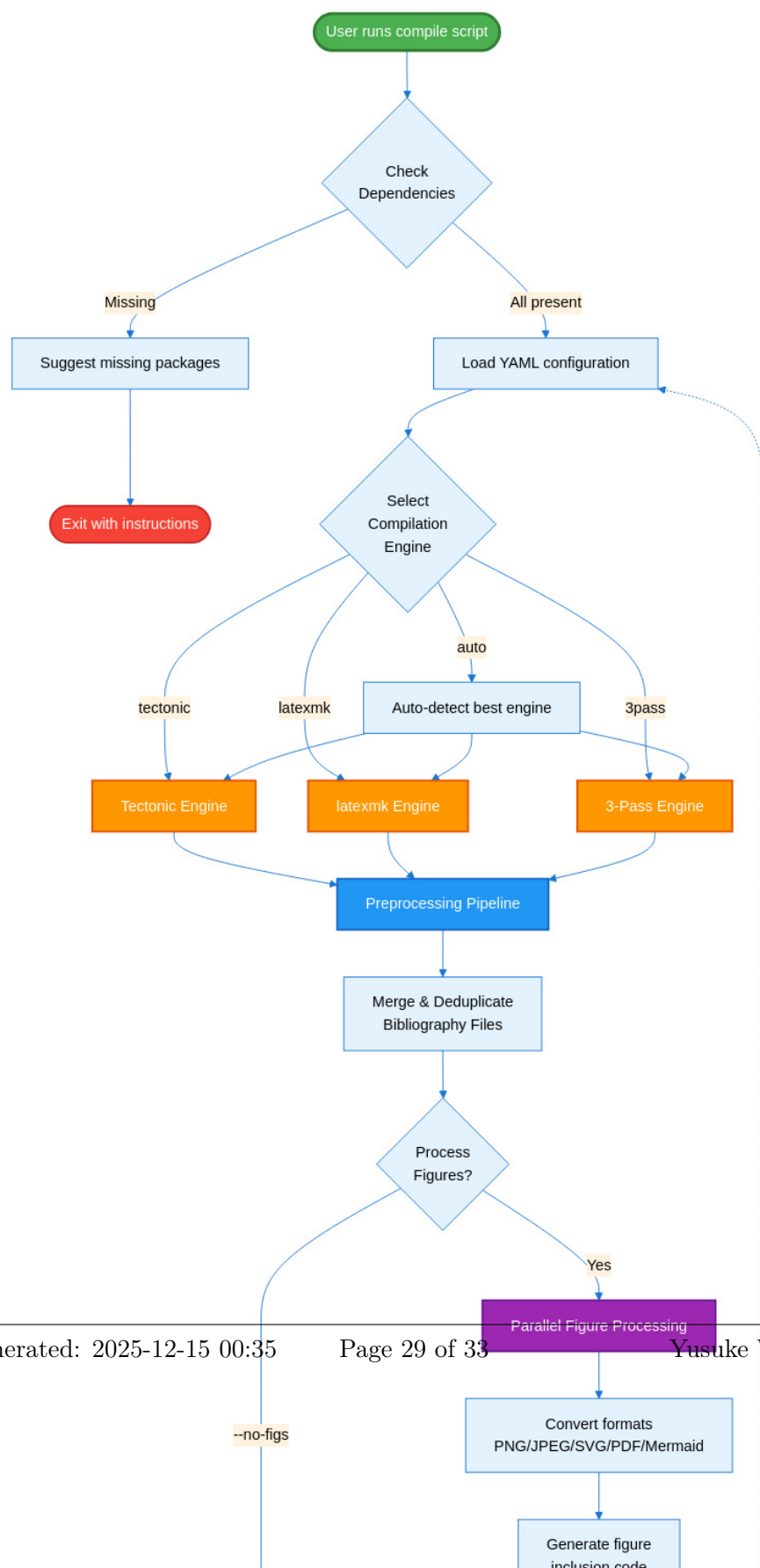




Figure 4 –

SciTeX Writer Directory

Organization. This diagram shows the hierarchical organization of the repository structure. The `00_shared/` directory (green) contains resources used across all document types, including bibliography files and LaTeX styles, ensuring consistency without duplication. The three document directories (blue/purple) follow identical internal structures, facilitating familiarity and code reuse. Each document has its own `contents/` subdirectory with `figures/` and `tables/` organized into `caption_and_media/` (source files) and `compiled/` (generated LaTeX code). The modular structure minimizes merge conflicts during collaborative editing by isolating frequently-modified content files. Compilation scripts (orange) and configuration files (red) reside in dedicated directories for easy discovery and maintenance. Dotted lines indicate that supplementary materials follow the same organizational pattern as the main manuscript.

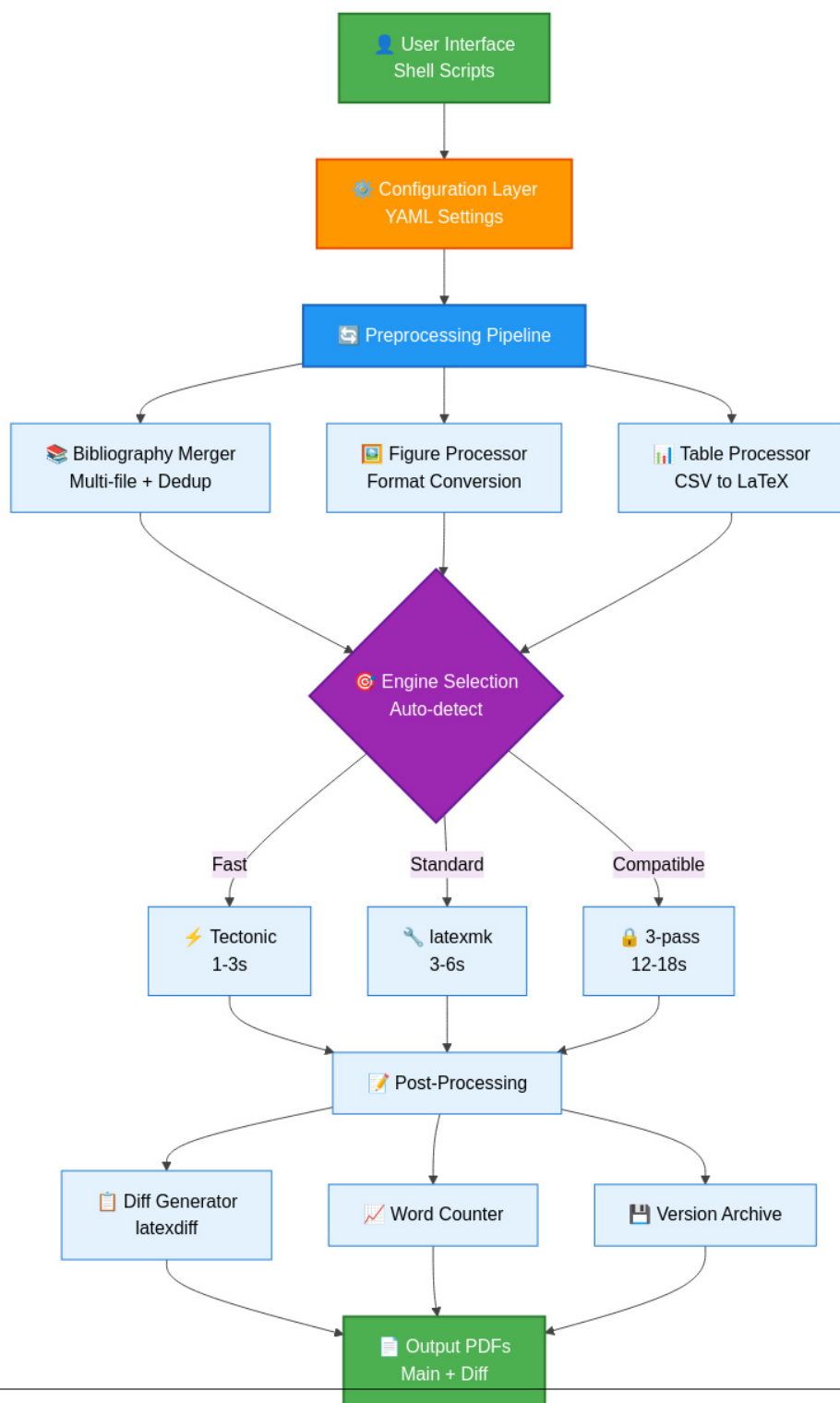


Figure 5 – SciTeX Writer System Architecture. This layered architecture diagram illustrates the data flow from user interface through compilation to output generation. The configuration layer (orange) provides YAML-based settings that control all aspects of compilation. The preprocessing pipeline (blue) handles bibliography merging, figure format

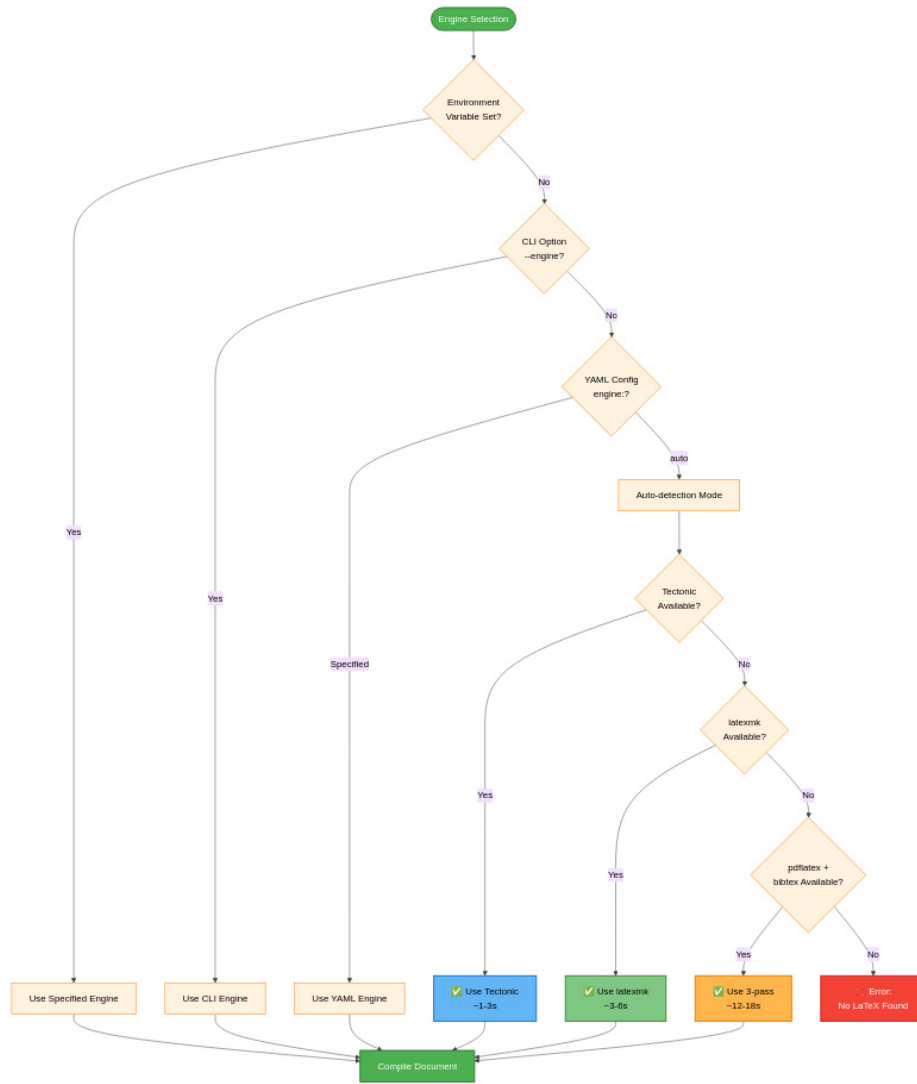


Figure 6 – Compilation Engine Selection Decision Tree.

The system prioritizes explicit user configuration over automatic detection. Environment variables provide the highest priority override (useful for CI/CD pipelines). Command-line arguments (`--engine`) override YAML configuration (useful for one-off compilations). When engine is set to 'auto' (default), the system attempts to use Tectonic first for speed, falling back to latexmk for reliability, and finally to 3-pass for maximum compatibility. This cascading detection ensures the system always finds an available compilation method while preferring faster engines when possible. Times shown are typical

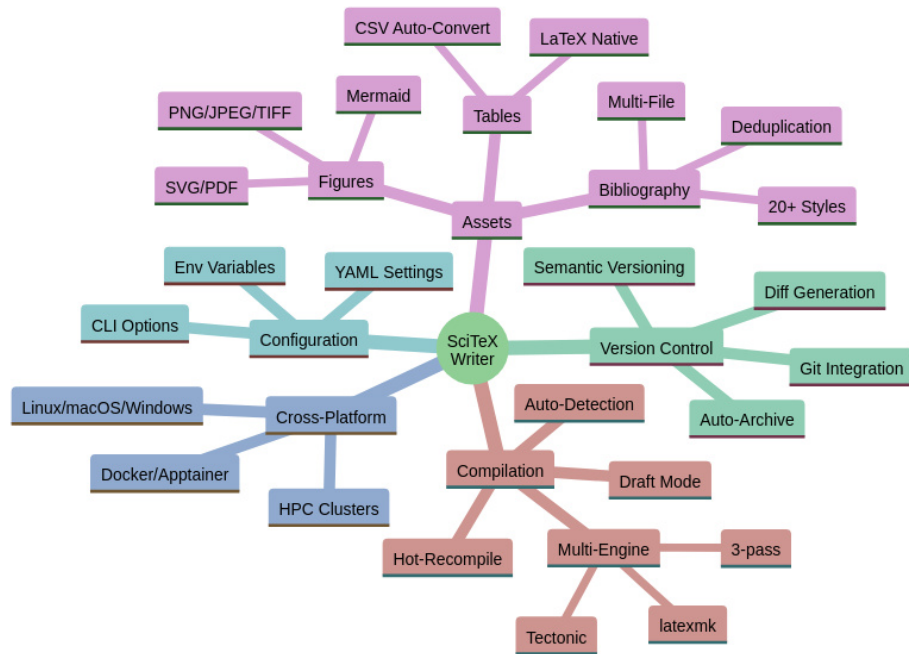


Figure 7 – SciTeX Writer Feature Overview. This mind map provides a comprehensive visualization of the framework’s major capabilities organized into five categories. Compilation features include multi-engine support with auto-detection and performance optimization modes. Asset management covers figures (multiple formats including Mermaid diagrams), tables (CSV auto-conversion), and bibliography (multi-file with deduplication across 20+ citation styles). Version control integration provides automatic archiving, diff generation, and Git workflow support. Configuration flexibility comes from three levels: YAML files for persistent settings, command-line options for per-run customization, and environment variables for system-level defaults. Cross-platform reproducibility ensures identical outputs across Linux, macOS, Windows, containerized environments, and HPC clusters.



Figure 8 — Multi-File Bibliography Processing and Deduplication. The bibliography system enables researchers to organize references across multiple topical bib files. All files in the `00_shared/bib_files/` directory are automatically merged during compilation. The deduplication engine implements a two-tier matching strategy: DOI-based matching provides exact duplicate detection when DOIs are present, while title and year matching handles entries without DOIs. This approach eliminates duplicate references that commonly appear when the same paper is cited across multiple source files. The final bibliography is sorted according to the selected citation style (by citation order for numbered styles, alphabetically for author-year styles). Color coding: blue (merging), purple (deduplication logic), green (output).